Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 04

Lecture – 19

Lec-19: VR Software

Hello and welcome to the NPTEL MOOCs course on Foundations for Virtual and Augmented Reality Systems. We are going to start our discussion on VR software as part of lecture number 10. What have we learned so far? Let us have a quick recap. So, in this part, in the previous few lectures, we have talked about VR hardware.

AR hardware and tracking hardware. So, those are all related to how the hardware, what kind of hardware is used for generating the VR experience or the AR experience or enhancing the experience through tracking using the tracking hardware. What we are going to cover now is going to be an introduction to the software part. which drives the hardware.

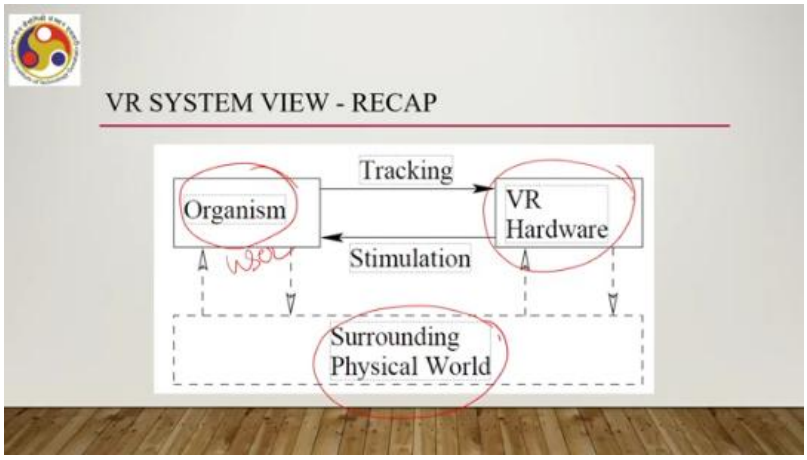
Now, when I say software, typically we know that software is meant to drive hardware. So, the role of software, what is the role? The obvious answer is to drive the hardware, but in the context of extended reality or rather VR or AR. this obvious answer may not be the right answer. So, the not so obvious answer can be to drive the hardware to generate experience utilizing our senses.

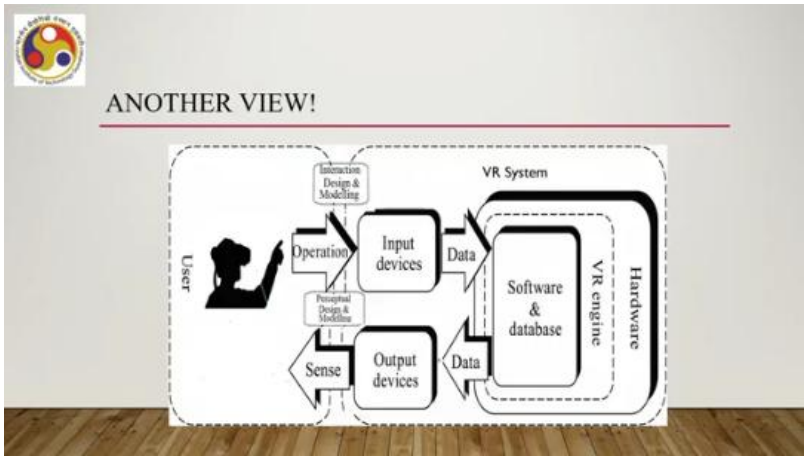
This is related to the definition of AR or VR. So, we can say that when we are talking about software in the context of VR or AR. We must ensure that the software is there to drive the hardware, VR hardware or AR hardware, to generate in this context a VR experience because we are talking about VR software utilizing our senses. That is the role of VR software. So, it must drive the hardware, but that is not sufficient.

Along with that, it should be able to drive the hardware in a way such that appropriate stimuli are generated and those stimuli are appropriately received to stimulate our sense organs. Now, when we talk of a VR system, if you can recollect from our earlier discussion

on VR hardware, we talked of VR system where we saw that there are three components: VR hardware, surrounding physical world, and the organism, which is nothing but the user, the VR user.

And between each of these components, there are bidirectional data movements; tracking and stimulation happen between the user and the hardware as shown here. Whereas there are data flows, bidirectional data flows, between the other between VR hardware, the surrounding world, as well as organism and surrounding world. Of primary concern to us here in our current discussion are the data flows between the user and the VR hardware. So, if we have a look at the data flow schematically, we can get an illustration like this as shown here, where on one end we have the user, and on the other end we have the hardware.





Now, what happens is that the user makes use of the input devices. So, the user operates; some operation takes place between the user and the input devices, which generates some data that is sent to the VR system. This data gets processed in the VR system and then that processed data is sent back to the output devices. which are there to generate suitable

stimuli which are supposed to affect our sense organs suitably as we have discussed before. Now, so there are three things.

One is when the user is interacting with the input devices, we need to have proper interaction design and modeling. That is one part. We also need to have perceptual design and modeling so that stimuli generated by the hardware by the output devices can suitably stimulate our sense organs. But also we need to have the software to process the input data and generate the output data. Now here in this diagram, as you can see, the software part we have mentioned as part of two components as two components: VR engine and the other component of the software that is software and database. We will go through these differences in the subsequent part of the lecture. So, that is the broader way of looking at the data flow between user and the system and where the software is placed. Now, when we say we wish to create a VR experience So, creating such an experience involves several activities, broadly three activities.

What are those? Creating the virtual environment, creating the stereoscopic view that is the output, and incorporating interactions. So, creation of virtual environment can also be termed as modeling of the environment or object modeling. Creation of the stereoscopic view that is the output which can also be termed as rendering.

Now, when I say view or rendering it does not only refer to visual senses. I am using the term in a very broad sense; it can refer to other senses as well, and the third one is incorporating interaction. So, that is the interaction part of the overall experience which involves tracking hardware. So, these three activities together lead to the VR experience. So, when we talk of VR software, the software should allow the user or should allow the developer—here the user is not the end user, rather the developer—to create a VR application.

In other words, the developer should be able to get assistance in all the three activities that are listed. That is from the developer's point of view. Of course, from the user's point of view, the user is not bothered about how the activities are done; rather, the user is bothered about how the end experience is. So, the user should get a very easy-to-use system to get the good experience.

Now let us focus primarily from the developer's point of view. What the developer expects ideally and what the developer gets in reality or in practice. So, ideal versus practical scenario, let us have a quick look at. So what is the developer's expectation? Ideally what the system or the software should allow the developer to do to do and in what manner to

do things. Ideally, a developer wants to provide only high-level descriptions; that is what a developer expects to do, and the software that supports the developer, to build the application should determine automatically all the low-level details from the high-level descriptions. So, there can be a middleware between the developer and the hardware. Developer follows well-known patterns; complicated details are avoided by calling functions from a well-designed software library.

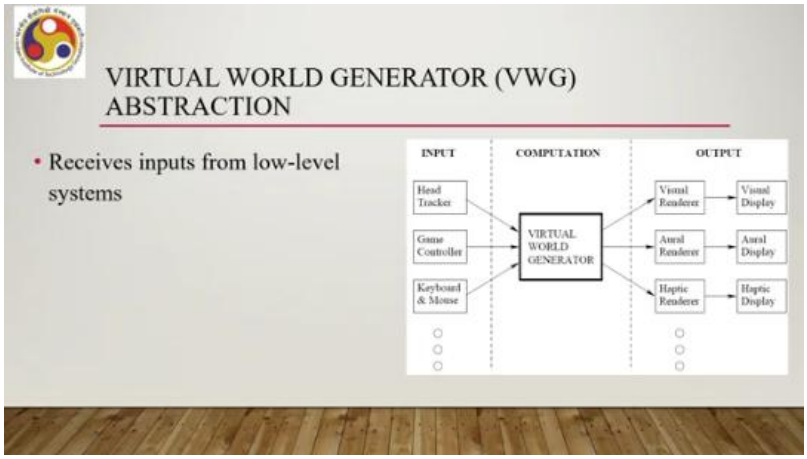
So, developer should have access to such libraries; the developer can follow the patterns and make use of the library calls rather than actually trying to do micro-level activities. However, that is using a middleware. Now, if the developer wants to do something original rather than depending on middleware support, the developer should also be able to design functions from scratch. That ideally should also be that support should also be there.

So, all these things are what the developer wants, but what is the reality that the developer gets in practice? Now, developer may need to implement much of the functionality from scratch rather than getting such support where the developer can only provide high-level descriptions and the system automatically converts that to low-level descriptions. may involve development activities may involve utilizing a software development kit or SDK for particular headsets that handles low-level operations such as device drivers, head tracking, and display output.

Here although we are using the term headset, we can keep in mind it is better to keep in mind that this is a broader term. Platform rather to say SDKs are platform dependent and for a given platform which can be a combination of hardware and OS. Lowest level operations sometimes the developer does not need to handle lowest level operations, but must understand an SDK that handles lowest level operations. Developer can use the SDK to deal with lowest level platform specific operations. Or alternatively can use a game engine which as we discussed earlier which is a middleware which has been specifically created or adapted for VR experience generation.

Such middleware can avoid substantial effort at first but may be cumbersome to implement ideas not part of the standard video games. So, generally such game engines as the name suggests are meant for developing games and those games need not be immersive VR games. So, such engines were developed for building any games, it can be adopted for VR application development or specific VR engines can be developed or created. If it is not specifically VR engine, then the developer loses some flexibility in terms of what he or she can do or cannot do.

So, what are the key concerns when we are talking of a VR software from the developer's point of view? We can understand that in terms of some key components that the software should support for a developer to build a proper VR experience. In order to understand that, we can think of the whole software system as an abstraction which is a virtual world generation or virtual world generator abstraction. We can call it VWG. The idea is schematically illustrated here in this accompanying figure.



We have this VWG here, which is nothing but a black box. At this stage, we do not know how things work inside. So, it receives input from low-level systems. That is on the left-hand side input. Such systems can be head tracker to estimate user's head position and orientation, keyboard, mouse, game controller, and other inputs.

And such input events are assumed to arrive in a queue ready to be processed. So, there can be multiple input events coming simultaneously, but those need to be arranged in a queue so that they can be processed systematically. So, we have input side: head tracker, game controller, keyboard, mouse, and many other inputs can be there. They arrive to this VWG kept in a queue to be processed. Now, the key role of the VWG is to maintain enough of an internal reality.

So, it should appear to be real to the user. Now, to do that, it should be able to maintain internal reality so that renderers can extract information to calculate outputs for respective displays. So, from the input side, whatever information is coming, the role of VWG is to convert it to output so that it appears to be real. So, there is a match between input and output. So, the role of VR VWG is to maintain that match.

Now, how can it be done? So, in order to maintain that match. So, the output devices have to be synced with the input data. So, we can have several types of output devices such as

visual renderer which generates visual display, aural renderer generates aural display to be sensed with our ears, haptic renderer generates haptic display to be sensed with our skin, and there can be many such other renderers as we have discussed during our VR hardware discussion. Now to maintain that match, VWG must support a few things. What are those things? First is virtual world. Now when we say that visual display renders a visual scene comprising a virtual world, that world may appear real or it can be synthetically created.

So virtual world can be completely synthetic. So that means it requires defining numerous triangles in a 3D space, essentially a triangular mesh we have to create. So later on when we talk about how to create such scenes as part of a discussion on computer graphics, we will understand what these triangular meshes are, why they are required, and how they are represented. However, it is sufficient at this stage to know that in order to be able to create a synthetic scene, we need to maintain such a mesh of triangles.

And not only that, along with that we need to maintain material properties that indicate how they interact with light, sound, forces and so on. So, the physical properties of the surfaces we also need to maintain. Now, these are typical problems in computer graphics. Computer graphics addresses computer-generated images from synthetic models, and the software that is meant to help the developer build an application should support computer graphics operations. So, that is the first requirement.

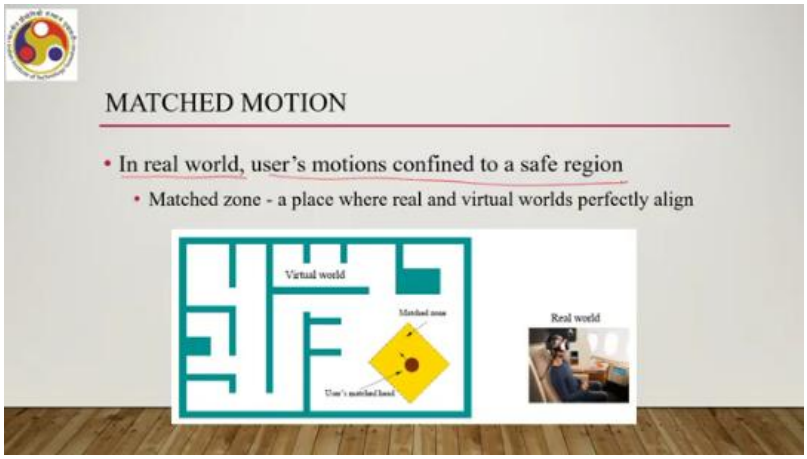
Alternatively, we can have a virtual world that is recorded physical world so the world is realistic. Now in order to do that, we can capture the real world using cameras and that relies on computer vision techniques and also as we have discussed earlier, simultaneous localization and mapping or SLAM techniques. So in order to create a real-looking virtual world, we may require support for such SLAM or computer vision in our software for the developer to utilize and create applications accordingly. Now, there can be other possibilities of creating a virtual world as well.

Many possibilities, in fact, exist in between. One such possibility can be that camera images may be taken of a real object and mapped onto a synthetic object in the virtual world. This is called texture mapping, which is a very common problem or operation in computer graphics. So our software ideally should support texture mapping operations as well. So that is about the creation of a virtual world.

Another thing is motion. Now there is something called matched motion. Let us try to understand what it is and what kind of support, in the software we should have. Now, the

basic operation of the VWG is to maintain correspondence between user motions in the real and virtual world, that is what a VWG is supposed to do.

Now, it may so happen that in the real world, users' motions are confined to a safe region. Which may not be the case in the virtual world. So, there is something called a matched zone that is a place where real and virtual worlds perfectly align. So, this is a part of the, say, virtual world, but the user can move within this space in the real world and this movement is translated to the other parts of the virtual world. So, this region where the user's physical movement is restricted and which is also a part of the virtual world is called a matched zone.



Now, when implementing an application, when implementing an immersive experience, if it is not carefully done, then there can be a mismatch of obstacles. Say, for example, a user can be blocked in the virtual world, but not in the real world. So, within that zone, there is no physical obstacle, but in the virtual world, there may be some virtual obstacle. What happens in such a case? Or vice versa, that is, there is a physical obstacle in the matched zone, whereas there is no such obstacle in the virtual world.

What happens in such a case? So, that is a challenge to deal with. Which motions from the real world should be reflected in the virtual world? That is how we can resolve such challenges by addressing these types of fundamental questions. Now, the motion that should be reflected in the virtual world varies among VR experiences.

Head motions must be matched so that the visual renderer uses the correct viewpoint in the virtual world. We have already discussed this before. Other body parts in such cases may be less critical, but sometimes they may be important. So, if a user needs to perform hand-eye coordination or looks at other parts of the body during the experience,

then the user expects the other parts also to move in accordance with the head movement or in accordance with the movement in the virtual world. So, in such cases, body part tracking and the corresponding matched motion with the head movement is required. In many VR experiences, users want to move well outside of the matched zone. Which motivates locomotion. So, moving oneself in the virtual world while motion is not matched in the real world.

That is, we have already briefly talked about what happens in such cases. For example, suppose you want to explore a virtual city while remaining seated in the real world. So there can be different ways to do that. You can pull up a menu, point to where you want to go, and the teleportation operation can be performed, which can send you to the desired location in the city although you have not made any physical movement. So, these types of issues in matched motion happen when you are

may be doing one kind of physical motion in the matched zone whereas there can be a different kind of virtual motion and how do I create a correspondence between the two. That is where the challenge is and that is where a typical VR software should have support for developers to address those challenges. Then user locomotion is another concern which is closely related to the earlier one. A popular option to move oneself in the virtual world by operating a game controller, mouse or keyboard is there. Say for example, by pressing buttons or moving knobs, the user in the virtual world could be walking, running, jumping, swimming. That is the avatar of the user in the virtual world can perform all these activities without moving physically. Can also climb. a board or a vehicle in virtual world and operate its controls to move yourself. That is another metaphor, vehicle metaphor can be used while the user is sitting. That is possible.

But these operations, although convenient, so user does not need to actually physically move. This may lead to cyber sickness. So, we have talked about these things. during our discussion on cyber sickness that there is a mismatch between the user's physical motion and virtual motion which may lead to cyber sickness, which increases with the exposure duration and also with other factors.

So, support for user locomotion is also important different sorts of locomotion so that sickness can be addressed. So, that support is also part should be part of the VR software. So with that, we have come to the end of this part of the lecture where we have briefly covered the idea of VR software: what the software is and what the key components are. Now for the key components, we have talked about a few key components that a software,

VR software, should support primarily from the developer's point of view. So there are a few more components which we need to discuss.

We will take it up in the next lecture along with The different software packages, tools, and SDKs that are available to build and deploy VR systems on different platforms. So we will continue our discussion on VR software in the next part of the lecture. Till then, thank you and goodbye. I hope you have enjoyed. Looking forward to meeting you in the next part of the lecture. Happy learning.